

Matriz de Trazabilidad de Requerimientos (RTM)

Proyecto: StockSync — Grupo 5

Fecha: 15/06/2026

Versión: v1.0

Resumen

REQ	Descripción	Estado	Tests
REQ-F01	Producto Padre + SKU Hijo por matriz de atributos		9 tests
REQ-F02	Kardex transaccional inmutable		8 tests
REQ-F03	Venta con validación atómica sin saldos negativos		5 tests
REQ-F04	Alerta de punto de reorden		2 tests
REQ-F05	RBAC (Admin/Operator)		15 tests
REQ-NF01	Aislamiento transaccional Read Committed		3 tests
REQ-NF02	Consultas SKU < 300ms P95		—

REQ-F01 — Producto Padre con generación automática de SKUs Hijo por matriz de atributos

El sistema debe permitir definir un producto genérico (Producto Padre) y generar automáticamente todos los SKU hijos a partir de combinaciones de atributos (talle, color, etc.).

Código fuente

Archivo	Líneas	Función/Clase	Rol
app/models/product.py	37–51	class Category	Categoría del producto
app/models/product.py	53–64	class AttributeType	Tipo de atributo (Talle, Color)

Archivo	Líneas	Función/Clase	Rol
app/models/product.py	67–86	class AttributeValue	Valor de atributo (S, M, Rojo)
app/models/product.py	89–113	class Product	Producto Padre (SKU base)
app/models/product.py	116–143	class ProductVariant	SKU Hijo con atributos
app/services/product_service.py	84–91	ProductService._build_sku()	Construye SKU: BASE-ATTR1-ATTR2
app/services/product_service.py	121–157	ProductService.generate_variants()	Generación por producto cartesiano (itertools.product)
app/services/product_service.py	95–119	ProductService.add_variant()	Agrega variante singular
app/services/product_service.py	53–62	ProductService.create_product()	Crea producto padre
app/api/v1/products.py	157–165	generate_variants()	POST /products/{id}/variants/generate
app/api/v1/products.py	102–108	create_product()	POST /products/
app/api/v1/products.py	147–154	add_variant()	POST /products/{id}/variants
app/schemas/product.py	137–145	class GenerateVariantRequest	Schema con attribute_combinations

Tests

Archivo	Función	Verifica
tests/test_product_service.py:185	test_build_sku_success	_build_sku concatena CAMISETA-S-ROJO
tests/test_product_service.py:197	test_add_variant_success	Creación de variante singular
tests/test_product_service.py:216	test_add_variant_duplicate	SKU duplicado → 400

Archivo	Función	Verifica
tests/ test_product_service. py:236	test_generate_variants_success	2 talles × 2 colores = 4 variantes generadas
tests/ test_product_service. py:259	test_tc13_generate_variants_omite_duplicados	TC-13: Omite combinaciones existentes
tests/ test_product_service. py:47	test_create_product_success	Crea producto con SKU base
tests/ test_product_service. py:65	test_create_product_duplicate_sku	SKU base duplicado → 400
tests/ test_api_products.py: 17	test_tc07_crear_producto_admin	TC-07: Creación por Admin → 201
tests/ test_api_products.py: 33	test_tc08_crear_producto_sku_duplicado	TC-08: SKU duplicado → 400

REQ-F02 — Kardex transaccional inmutable con timestamp, usuario, tipo y cantidad

Cada movimiento de stock se registra en una tabla append-only. Nunca se hace UPDATE ni DELETE. El stock actual se calcula como SUM(quantity) de todos los movimientos de una variante.

Código fuente

Archivo	Líneas	Función/Clase	Rol
app/models/ stock.py	27–33	class MovementType	Enum: PURCHASE, SALE, ADJUSTMENT_IN, ADJUSTMENT_OUT, RETURN_IN, RETURN_OUT
app/models/ stock.py	36–70	class StockMovement	Modelo Kardex (append-only)
app/services/ stock_service.py	44–54	StockService.get_current_stock()	SUM(quantity) sobre movimientos
app/services/ stock_service.py	56–67	StockService.get_stock_summary()	Resumen con current_stock, below_reorder

Archivo	Líneas	Función/Clase	Rol
app/services/stock_service.py	69–78	StockService.get_movements()	Historial ordenado por fecha
app/services/stock_service.py	129–189	StockService.create_sale()	Genera movimiento SALE (quantity negativo)
app/services/stock_service.py	228–257	StockService.receive_purchase_order()	Genera movimiento PURCHASE
app/services/stock_service.py	102–123	StockService.adjust_stock()	Genera movimiento ADJUSTMENT_IN/OUT
app/api/v1/stock.py	28–35	get_stock_summary()	GET /stock/summary/{variant_id}
app/api/v1/stock.py	38–46	get_movements()	GET /stock/movements/{variant_id}
app/schemas/stock.py	14–26	class StockMovementResponse	Schema del Kardex

Tests

Archivo	Función	Verifica
tests/test_stock_service.py:59	test_get_current_stock_initial	Stock inicial = 0
tests/test_stock_service.py:69	test_adjust_stock_success	Ajuste +15 → stock 15
tests/test_stock_service.py:199	test_tc16_stock_suma_múltiples_movimientos	TC-16: +100 PURCHASE, -30 SALE, +10 ADJ = 80
tests/test_stock_service.py:225	test_tc17_stock_summary_variante_inexistente	TC-17: Variante 99999 → 404
tests/test_stock_service.py:324	test_tc25_venta_genera_movimiento_kardex_negativo	TC-25: Sale genera StockMovement qty=-10
tests/test_stock_service.py:358	test_tc26_recibir_order_ya_recibida	TC-26: Orden ya recibida → 400
tests/	test_tc27_recepcion_order	TC-27: Recepción genera

Archivo	Función	Verifica
test_stock_service.py:379	rden_genera_movimiento_purchase	PURCHASE qty=50
tests/test_api_stock.py:47	test_tc22_venta_exitosa_descuenta_stock	TC-22: Stock 20 → venta 5 → stock 15

REQ-F03 — Venta con validación de disponibilidad antes del commit, sin saldos negativos

Al registrar una venta, el sistema debe validar que hay stock suficiente antes de descontar. Si no hay stock, se rechaza la operación (HTTP 409) y se hace rollback.

Código fuente

Archivo	Líneas	Función/Clase	Rol
app/models/stock.py	127–147	class Sale	Modelo de venta
app/models/stock.py	150–164	class SaleItem	Item de venta con variant_id, quantity, unit_price
app/services/stock_service.py	129–189	StockService.create_sale()	Validación + bloqueo SELECT FOR UPDATE
app/services/stock_service.py	138–148	with_for_update()	Bloqueo de fila para evitar race conditions
app/services/stock_service.py	156–165	Validación stock	Si stock < cantidad → rollback + 409
app/api/v1/stock.py	76–83	create_sale()	POST /stock/sales
app/schemas/stock.py	49–58	class SaleItemCreate	Valida quantity > 0
app/schemas/stock.py	61–63	class SaleCreate	Schema de solicitud

Tests

Archivo	Función	Verifica
tests/test_stock_service.py:87	test_create_sale_insufficient_stock	Stock 0, vender 5 → 409

Archivo	Función	Verifica
tests/ test_stock_service.py :105	test_create_sale_success	Stock 10, vender 3 → saldo 7
tests/ test_stock_service.py :324	test_tc25_venta_genera_movimiento_kardex_negativo	TC-25: Sale genera movimiento qty=-10
tests/ test_api_stock.py:47	test_tc22_venta_exitosa_descuenta_stock	TC-22: Stock 20 → venta 5 → stock 15
tests/ test_api_stock.py:65	test_tc24_venta_quantity_cero_es_invalido	TC-24: qty=0 → 422

REQ-F04 — Alerta de punto de reorden cuando stock ≤ umbral definido por Admin

El Admin puede definir un punto de reorden por variante. El sistema alerta cuando el stock actual es menor o igual a ese umbral.

Código fuente

Archivo	Líneas	Función/Clase	Rol
app/models/ product.py	129	ProductVariant. reorder_point	Campo reorder_point (default=5)
app/services/ stock_service.py	80–96	StockService.get_low_stock_alerts()	Itera variantes, compara stock vs reorder_point
app/services/ stock_service.py	56–67	StockService.get_stock_summary()	Incluye flag below_reorder
app/api/v1/ stock.py	49–55	get_low_stock_alerts()	GET /stock/alerts/low-stock
app/schemas/ stock.py	36–42	class StockSummary	Campo below_reorder: bool

Tests

Archivo	Función	Verifica
tests/ test_stock_service.py :236	test_tc18_alerta_stock_igual_a_reorder_point	TC-18: stock=5, reorder=5 → aparece en alertas

Archivo	Función	Verifica
tests/ test_stock_service.py :268	test_tc19_sin_alerta_ stock_sobre_reorder_p oint	TC-19: stock=6, reorder=5 → NO aparece

REQ-F05 — RBAC: ajustes de inventario y eliminación lógica bloqueados para no-Admin

Solo los usuarios con rol Admin pueden crear/editar productos, ajustar stock, gestionar usuarios y confirmar recepciones. Los Operadores solo pueden ver y vender.

Código fuente

Archivo	Líneas	Función/Clase	Rol
app/models/ user.py	14–16	class UserRole	Enum ADMIN / OPERATOR
app/models/ user.py	19–42	class User	Campo role, propiedad is_admin
app/core/ security.py	59–73	get_current_use r()	Extrae JWT, obtiene usuario
app/core/ security.py	76–85	require_admin()	Si role ≠ ADMIN → 403
app/core/ security.py	36–41	create_access_t oken()	Crea JWT
app/services/ product_service .py	72–76	ProductService. delete_product()	Eliminación lógica (is_active = False)
app/services/ stock_service.p y	102–123	StockService.ad just_stock()	Ajuste de stock (Admin only)
app/api/v1/ products.py	133–141	delete_product()	Con Depends(require _admin)
app/api/v1/ products.py	102–108	create_product()	Con Depends(require _admin)
app/api/v1/ products.py	123–130	update_product()	Con Depends(require _admin)
app/api/v1/ products.py	147–154	add_variant()	Con Depends(require _admin)

Archivo	Líneas	Función/Clase	Rol
app/api/v1/products.py	157–165	generate_variants()	Con Depends(require_admin)
app/api/v1/stock.py	62–69	adjust_stock()	Con Depends(require_admin)
app/api/v1/stock.py	99–106	receive_purchase_order()	Con Depends(require_admin)
app/api/v1/users.py	14–21	list_users()	Con Depends(require_admin)
app/api/v1/users.py	23–30	create_user()	Con Depends(require_admin)

Tests

Archivo	Función	Verifica
tests/test_api_products.py:52	test_tc09_crear_producto_rol_operator	TC-09: Operator crea producto → 403
tests/test_api_products.py:78	test_tc11_eliminar_producto_logico	TC-11: Delete lógico por Admin → 204
tests/test_product_service.py:121	test_delete_product_success	is_active pasa a False
tests/test_product_service.py:139	test_delete_product_not_found	Producto inexistente → 404
tests/test_stock_service.py:300	test_tc21_ajuste_stock_negativo_genera_adjustment_out	TC-21: Ajuste -5 → ADJUSTMENT_OUT
tests/test_security.py:21	test_tc28_hash_password_produce_hash_bcrypt	TC-28: Hash bcrypt generado
tests/test_security.py:32	test_tc29_verify_password_correcto	TC-29: Verify correcto → True
tests/test_security.py:42	test_tc30_verify_password_incorrecto	TC-30: Verify incorrecto → False
tests/test_security.py:51	test_tc31_create_y_decode_access_token	TC-31: JWT encode/decode
tests/	test_tc32_decode_token	TC-32: Token inválido → 401

Archivo	Función	Verifica
test_security.py:62	n_invalido	
tests/ test_api_auth.py:26	test_tc01_login_credenciales_correctas	TC-01: Login OK → 200 + token
tests/ test_api_auth.py:42	test_tc02_login_password_incorrecto	TC-02: Password mal → 401
tests/ test_api_auth.py:56	test_tc03_login_usuario_inexistente	TC-03: Email inexistente → 401
tests/ test_api_auth.py:68	test_tc04_get_me_sin_token	TC-04: Sin token → 401
tests/ test_api_auth.py:77	test_tc05_get_me_con_token_valido	TC-05: Token válido → 200 + role

REQ-NF01 — Aislamiento transaccional Read Committed, prevención de lecturas sucias

Las transacciones de venta deben ejecutarse con aislamiento Read Committed. El sistema debe prevenir lecturas sucias y condiciones de carrera sobre el stock.

Código fuente

Archivo	Líneas	Función/Clase	Rol
app/services/ stock_service.py	129–189	StockService.create_sale()	Transacción con SELECT FOR UPDATE
app/services/ stock_service.py	140–148	with_for_update()	Bloqueo de fila PostgreSQL
app/services/ stock_service.py	157–165	Rollback	db.rollback() si stock insuficiente
app/core/ database.py	10–11	create_engine()	PostgreSQL, por defecto Read Committed
app/core/ database.py	17	SessionLocal(autocommit=False)	Control manual de transacciones

Tests

Archivo	Función	Verifica
tests/ test_stock_service.py	test_create_sale_insufficient_stock	Rollback + 409 en stock insuficiente

Archivo	Función	Verifica
:87		
tests/ test_stock_service.py :105	test_create_sale_success	Transacción completa: venta + descuento
tests/ test_api_stock.py:47	test_tc22_venta_exitosa_descuenta_stock	TC-22: Flujo completo de venta

Nota: No hay test de concurrencia (ej: dos hilos vendiendo simultáneamente). Pendiente para próxima iteración.

REQ-NF02 — Consultas de disponibilidad de SKU < 300ms en percentil 95

Las consultas de stock actual de una variante deben responder en menos de 300ms en el percentil 95 de las mediciones.

Código fuente

Archivo	Líneas	Función/Clase	Rol
app/services/ stock_service.py	44–54	StockService.get_current_stock()	SUM(quantity) sobre movimientos
app/models/ stock.py	47	variant_id = Column(..., index=True)	Índice en variant_id
app/models/ stock.py	61	created_at = Column(..., index=True)	Índice en created_at
app/models/ product.py	101	base_sku = Column(..., unique=True, index=True)	Índice único en base_sku
app/models/ product.py	126	sku = Column(..., unique=True, index=True)	Índice único en variant.sku
app/models/ user.py	26	email = Column(..., index=True)	Índice en email

Tests

No hay tests de performance específicos para este requerimiento. Los índices en las columnas

consultadas (`variant_id` en `StockMovement`, `sku` en `ProductVariant`) optimizan las queries.

Pendiente: Agregar test de benchmark con `time.perf_counter()` que verifique que `get_current_stock()` responde en $< 300\text{ms}$ para N variantes.

Cobertura de pruebas por requerimiento

REQ	Tests	Archivos de test
REQ-F01	9	<code>test_product_service.py</code> , <code>test_api_products.py</code>
REQ-F02	8	<code>test_stock_service.py</code> , <code>test_api_stock.py</code>
REQ-F03	5	<code>test_stock_service.py</code> , <code>test_api_stock.py</code>
REQ-F04	2	<code>test_stock_service.py</code>
REQ-F05	15	<code>test_security.py</code> , <code>test_api_auth.py</code> , <code>test_api_products.py</code> , <code>test_product_service.py</code> , <code>test_stock_service.py</code>
REQ-NF01	3	<code>test_stock_service.py</code> , <code>test_api_stock.py</code>
REQ-NF02	0	—
Total	42	7 archivos